

Research use only — not a medical device. This software is for research and engineering experimentation. It is not validated for clinical or diagnostic use. The tooth-root cutoff is a heuristic (or a model trained on heuristic/annotated labels) and should be verified by a qualified person before any real-world use.

No patient data or trained weights are included in this repository (see `.gitignore`). You supply your own CBCT scans and models locally.

Credits

Built on [nnU-Net v2](#) for segmentation and [DentalSegmentator](#) for the teeth/bone model. Please cite those projects if you use this work.

CBCT Dominant-Arch & Tooth-Root Pipeline

Segment a CBCT volume, keep only the **dominant arch** (upper *or* lower, whichever has more bone), and write two masks: the arch bone, and the tooth **roots** (each tooth cut at the alveolar bone boundary, crowns removed).

With DentalSegmentator, a lower-dominant scan gives:

```
<patient>_Mandible.nii.gz      # the dominant arch bone
<patient>_teeth.nii.gz         # full teeth (crown + root)
<patient>_trabecular.nii.gz     # teeth roots only (crowns cut off)
```

An upper-dominant scan names the bone file `<patient>_Maxilla & Upper bone.nii.gz` instead; `teeth` and `trabecular` keep the same names. The non-dominant arch is discarded. The bone filename comes from `arches.*.name`; `teeth/trabecular` names come from `output.teeth_name` / `output.root_name`.

How it works, configurably:

- **Dominant arch** — `arches.{upper,lower}` map labels to each arch; the one with the larger `dominance.metric` (bone / teeth / total) wins.
- **Root extraction** — the root is the part of each tooth the alveolar bone *wraps around*, decided slice by slice (per-slice hole-fill + a small closing). The crown/neck float above the bone, so they're excluded, and the root top lands at the bone surface (seamless, never above it). `bone_close_mm` bridges thin gaps; `smooth_iter` cleans the edge.

Root extraction is a geometric heuristic, not a learned model. It assumes teeth erupt roughly along the S-I axis (true for standard CBCT). Eyeball a few cases.

Pipeline

```
.nii/.nii.gz -> DentalSegmentator/nnU-Net -> pick dominant arch -> cut teeth at
                                     bone + teeth labels      (discard other)      crest -> roots
```

Install

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

`nnunetv2` pulls in PyTorch. If you need a specific CUDA build, install torch first from <https://pytorch.org>, then `pip install -r requirements.txt`.

Run the app (UI)

```
streamlit run app.py
```

Opens a local web UI (default <http://localhost:8501>). You can:

- **Add multiple patients** — point at a folder of `.nii/.nii.gz` files, or upload several at once. They collect into a patient queue.
- Set the model preset, `model_dir`, device, folds, dominance metric, and root options in the sidebar.
- **Run the batch** with a per-patient progress bar; failures don't stop the run.
- Review each patient's three output files (with download buttons) and a colour-coded slice overlay (bone / teeth / root) for QC.

For testing without a GPU, tick **Skip inference — use existing labelmaps** and point it at a folder of labelmaps (matched to patients by id in the filename).

Run as an app (double-click)

No command line needed:

- **Windows:** double-click `run_app.bat`. It launches the app and opens your browser. If your packages live in a named conda env, open `run_app.bat` and set `CONDA_ENV=your_env_name` at the top.
- **macOS:** double-click `run_app.command` (first time: `chmod +x run_app.command`).
- **Linux:** run `./run_app.sh`.

All of these just run `python launch.py`, which starts the server on a free port, waits until it's ready, and opens the browser. Close the console window to stop.

Optional: build a .exe icon

Run `build_exe.bat` *inside the environment where you installed the requirements*. It uses PyInstaller to produce `dist\CBCCT_App.exe` — a thin launcher you can copy into the project folder and double-click. It does **not** bundle PyTorch/nnU-Net (those are large and stay in your environment); the .exe starts Streamlit using that environment's Python, so run it from a context where `python` resolves to that environment.

Get the model — DentalSegmentator (recommended, has ready weights)

DentalSegmentator ships pretrained nnU-Net v2 weights (unlike ToothFairy2, which you'd have to train yourself). 5 classes: upper skull, mandible, upper teeth, lower teeth, mandibular canal. This project runs its inference **in-process** via the nnU-Net Python predictor API — no Slicer, no PATH setup.

1. Download the weights (either source — same model):
 - Zenodo (for code use): <https://doi.org/10.5281/zenodo.10829675>
 - or [Dataset111_453CT_v100.zip](#) from <https://github.com/gaudot/SlicerDentalSegmentator/releases>
2. Extract so the trained-model folder exists:


```
...\Dataset111_453CT\nnUNetTrainer__nnUNetPlans__3d_fullres\
```

 (it must contain `plans.json`, `dataset.json` and `fold_*` with `checkpoint_final.pth`)
3. Set `nnunet.model_dir` in `config.dentalsegmentator.yaml` to that folder, then:

```
python run.py -i data/0102871823.nii.gz -o out/ -c
config.dentalsegmentator.yaml
```

Check which `fold_*` dirs exist in that folder and set `nnunet.folds` accordingly (`all`, `0` `1` `2` `3` `4`, or `0`). Requires `torch` (install a CUDA-matched build from <https://pytorch.org> for GPU; CPU works but is slow).

Get the model — ToothFairy2 (per-tooth FDI, train-it-yourself)

Download the ToothFairy2 nnU-Net weights (challenge config + instructions live in the nnU-Net repo under [documentation/competitions/Toothfairy2](#)). Place them so that `nnUNet_results/Dataset112_*/...` exists, then either:

- set `nnunet.results_dir` in `config.yaml`, or
- `export nnUNet_results=/path/to/nnUNet_results`

Set `nnunet.dataset_id` to match your downloaded dataset (e.g. `112`).

Label IDs must match your checkpoint. The defaults in `config.yaml` follow the official ToothFairy2 scheme (jawbone = 1,2; permanent teeth = FDI 11–48). If your checkpoint's `dataset.json` differs, edit `labels.bone_labels` / `labels.teeth_labels`.

Run

```
# single volume (filename stem -> patient id, e.g. 123.nii.gz -> "123")
python run.py -i data/123.nii.gz -o out/

# whole folder
python run.py -i data/ -o out/

# reuse an existing labelmap, skip nnU-Net (no GPU/weights needed)
python run.py -i data/123.nii.gz -o out/ --labelmap labels/123_seg.nii.gz

# force a patient id
python run.py -i scan.nii.gz -o out/ --patient-id 123
```

Verify without weights

```
python tests/test_synthetic.py
```

Builds a synthetic lower-dominant scan with crowns above the crest, runs the full pipeline, and checks that only the mandible arch is output, the upper arch is discarded, crowns are removed, and roots below the crest are kept.

Train a root model (optional, replaces the heuristic)

If you have root ground-truth labels, you can train an nnU-Net that learns the crown/root cutoff directly instead of using the geometric heuristic. Design A keeps DentalSegmentator for teeth+bone and trains a small model that takes **CBCT + tooth + bone** channels and outputs the root. Build the dataset with [training/prepare_dataset.py](#), then follow [training/TRAINING.md](#) for the nnU-Net commands. Once trained it drops into the pipeline in place of **extract_root**.

Layout

app.py	Streamlit UI (multi-patient batch + previews)
run.py	CLI entry point
config.dentalsegmentator.yaml	DentalSegmentator (ready weights) config
config.yaml	ToothFairy2 config
cbct_pipeline/	
io_utils.py	load/save NIfTI, preserve affine
labels.py	label maps, label -> mask extraction
segmentation.py	nnU-Net inference (python predictor + CLI backends)
roots.py	alveolar-crest estimation + tooth-root cut
preview.py	slice-overlay QC PNGs
pipeline.py	dominant-arch selection + orchestration
tests/test_synthetic.py	no-GPU smoke test

([trabecular.py](#) is retained for reference but no longer used by the pipeline; the cortical/trabecular split was superseded by root extraction.)

Notes / next steps

- Upper and lower teeth are solid masks (DentalSegmentator does not number teeth). For per-tooth FDI labels use the ToothFairy2 route.
- Verify label numbers from your checkpoint match the **arches.*** label IDs.
- The dominant-arch rule discards a whole arch; on a balanced full-FOV scan the two arches may be close, so check **dominance.metric** suits your data.